

Курс DevOps

Лабораторная работа №1

GitHub webhooks

Дата: 17 августа 2025 г.

17 августа 2025 г.

Общие инструкции

Рекомендации к выполнению

- Настоятельно рекомендуется выполнять команды из листинга отдельно, одну за другой.
- Рекомендуется копировать команды через промежуточную вставку в текстовый редактор во избежание лишних символов.

Требования к оформлению

- Все команды должны быть задокументированы с объяснением их назначения
- Скриншоты результатов выполнения команд обязательны
- Код должен быть оформлен с использованием синтаксической подсветки
- Обязательно указывать версии используемого ПО

Структура отчета

1. Цель работы
2. Задания с решениями
3. Результаты выполнения
4. Выводы

Критерии оценки

- **Отлично (5):** Все задания выполнены правильно, отчет оформлен согласно требованиям
- **Хорошо (4):** Большинство заданий выполнено правильно, незначительные недочеты в оформлении
- **Удовлетворительно (3):** Основные задания выполнены, есть ошибки или недочеты
- **Неудовлетворительно (2):** Задания выполнены неправильно или не выполнены

1 Цель работы

Научиться использовать GitHub webhooks для решения задач автоматизации сборки приложения и его развертывания.

2 Содержание работы

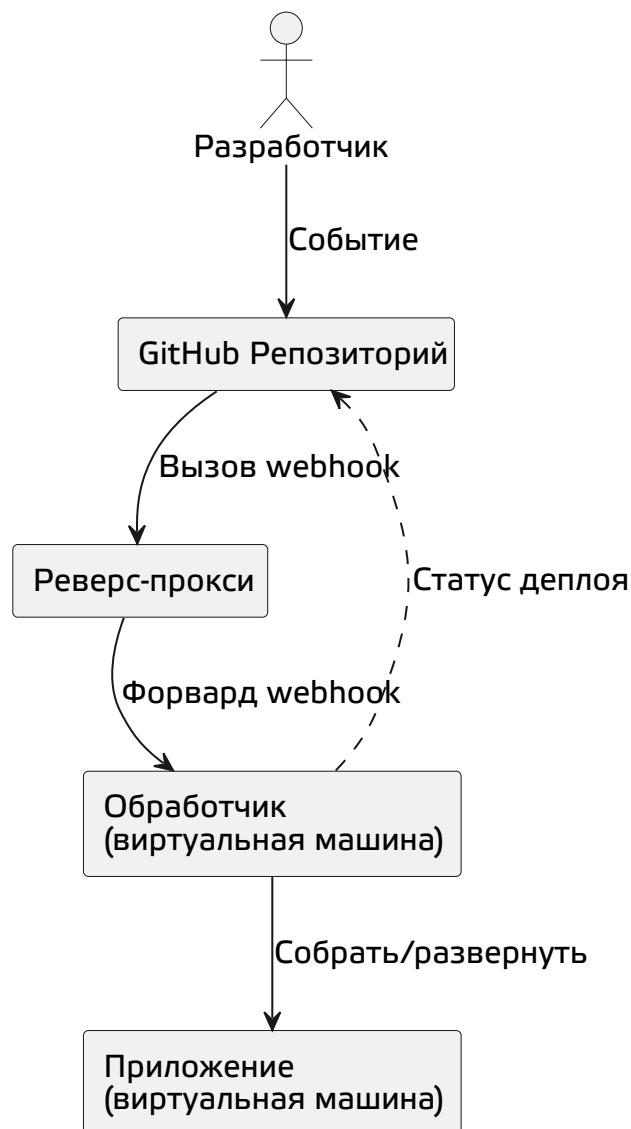


Рис. 1: Схема работы GitHub webhooks

В данной работе мы изучим механизм GitHub webhooks — автоматических HTTP запросов, которые отправляются при определенных событиях в репозитории. Как показано на схеме выше, процесс работы состоит из следующих этапов:

1. **Событие в репозитории:** разработчик выполняет push, создает pull request или другое действие

2. **Активация webhook:** GitHub автоматически отправляет HTTP POST-запрос на настроенный URL
3. **Обработка через прокси:** реверс-прокси перенаправляет запрос к обработчику
4. **Автоматическое развертывание:** обработчик получает webhook и запускает процесс сборки/развертывания приложения
5. **Обратная связь:** система может отправить статус выполнения обратно в GitHub

Этот подход позволяет использовать Git-репозиторий не только как средство совместной работы с кодом, но и как центральный элемент автоматизации процессов разработки и развертывания ПО, что является ключевой практикой DevOps.

2.1 Задание 1

Настроить виртуальную машину с Ubuntu 24.

2.2 Задание 2

На виртуальной машине настроить доступ до прокси сервера frp.

```
# Настройка переменных. Актуальные значения узнайте у преподавателя!  
PROXY=course.prafdin.ru  
TOKEN=pravdin2001  
ID=prafdin
```

```
# Скачать и установить frp как systemd сервисы.  
# Команда запросит пароль текущего пользователя  
wget -qO- https://gist.github.com/lawrenceching/41244a182307940cc15b45e3c4997346/raw/0576ea85d898c965c3137f7c38f9815e1233e0d1  
install-frp-as-systemd-service.sh | sudo bash
```

```
# Настройка доступа до frp server.  
# Команда запросит пароль текущего пользователя  
sudo tee /etc/frp/frpc.toml > /dev/null <<EOF # Начало команды  
serverAddr = "$PROXY"  
serverPort = 7000
```

```
auth.method = "token"  
auth.token = "$TOKEN"
```

```
[[proxies]]  
name = "hook-$ID"  
type = "http"  
localPort = 8080  
customDomains = ["webhook.$ID.$PROXY"]
```

```
[[proxies]]  
name = "app-$ID"  
type = "http"
```

```
localPort = 8181
customDomains = ["app.$ID.$PROXY"]
EOF # Конец команды

# Запустите frp client. Команда запросит пароль текущего пользователя
sudo systemctl status frpc

# Проверьте корректность настройки
echo "Адрес для проверки: webhook.$ID.$PROXY"

wget -qO- https://gist.githubusercontent.com/
prafdin/b9ff40c8bc6dc8c55ca7ac911e278ecc\
/raw/ea80e83e6a36220e2e9ccdaca8fde49ee888f2ab\
/main.py | python3

# Перейдите по адресу для проверки в браузере.
# Если всё настроено правильно, то в браузере получите ok
```

2.3 Задание 3

1. **Easy:** задеплоить статический сайт с помощью `nginx`. Событие - любой коммит в `main` ветку.
2. **Medium:** задеплоить новую версию `python web` приложения. Событие - создание любого тега. Выставить статус деплоя на коммит тега.
3. **Hard:** собрать `Java/C Sharp/C++ web` приложение, задеплоить новую версию приложения. Событие - создание `GitHub Release`. Выставить статус деплоя на коммит релиза.

Все необходимые зависимости приложения предварительно могут быть установлены вручную на виртуальной машине.

При необходимости используйте шаблон `python` модуля для запуска обработчика `webhook` <https://github.com/prafdin/SampleAppUpdateModule.git>

В качестве приложений можете создать форк следующих проектов:

1. Статический сайт <https://github.com/cloudacademy/static-website-example.git>
2. `Python` приложение TBD
3. `Java` приложение TBD

3 Контрольные вопросы

1. TBD

4 Выводы

TBD